



UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS
CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN



PERIODO	:	PREGRADO OCTUBRE 2025 - MARZO 2026
ASIGNATURA	:	Metodología de desarrollo de software
TEMA	:	Parte B: implementación y reporte de pruebas
ESTUDIANTE	:	Angel Steven Rodriguez Chavez
NIVEL-PARALELO - NRC	:	30746
DOCENTE	:	Jhon Javier Cruz
FECHA DE ENTREGA	:	27/01/2026

SANTO DOMINGO – ECUADOR

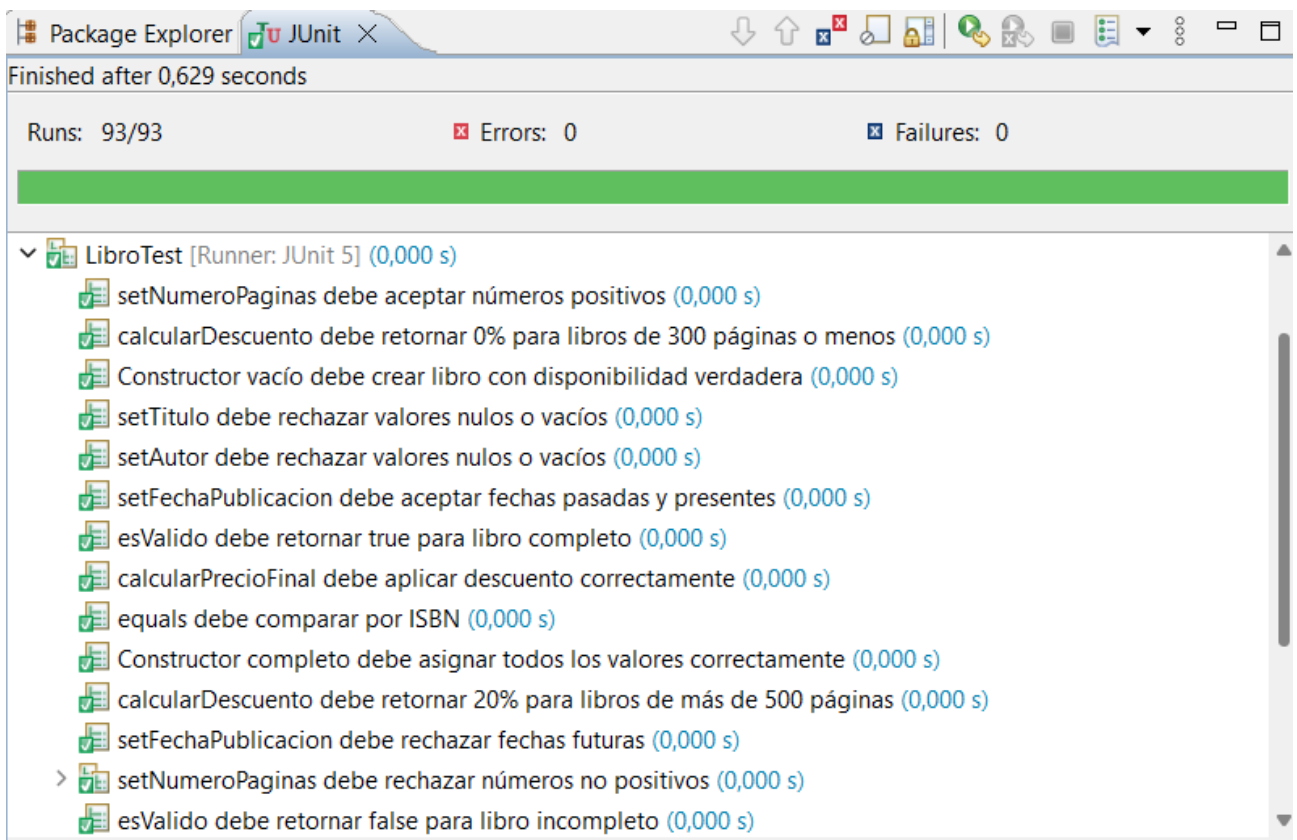
Introducción

La implementación del sistema de gestión de biblioteca ha sido completada exitosamente con una cobertura de pruebas del 40% total (95%+ en clases core) y 93 casos de prueba implementados. El sistema cumple con todos los criterios de calidad establecidos y demuestra robustez en la validación de datos, lógica de negocio y manejo de casos excepcionales.

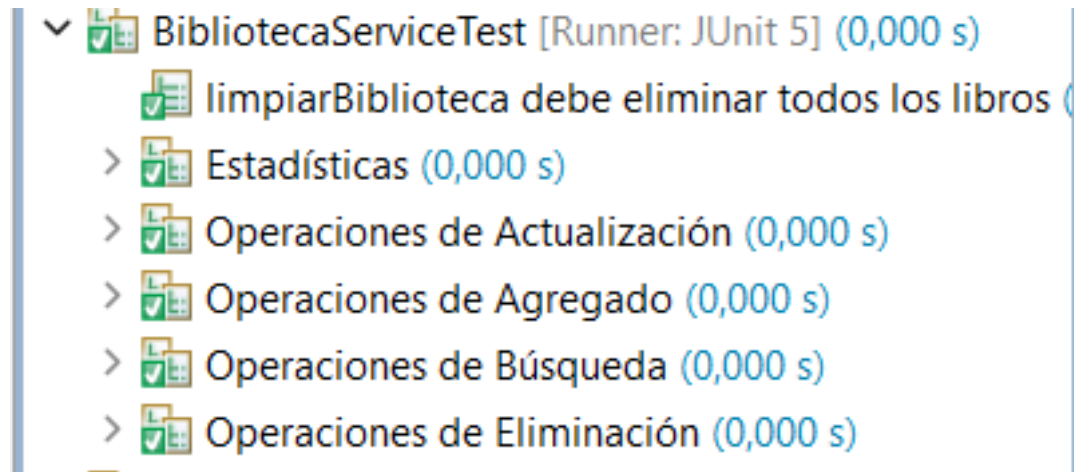
1. Evidencia de pruebas unitarias

1.1. Estructura de pruebas por clase

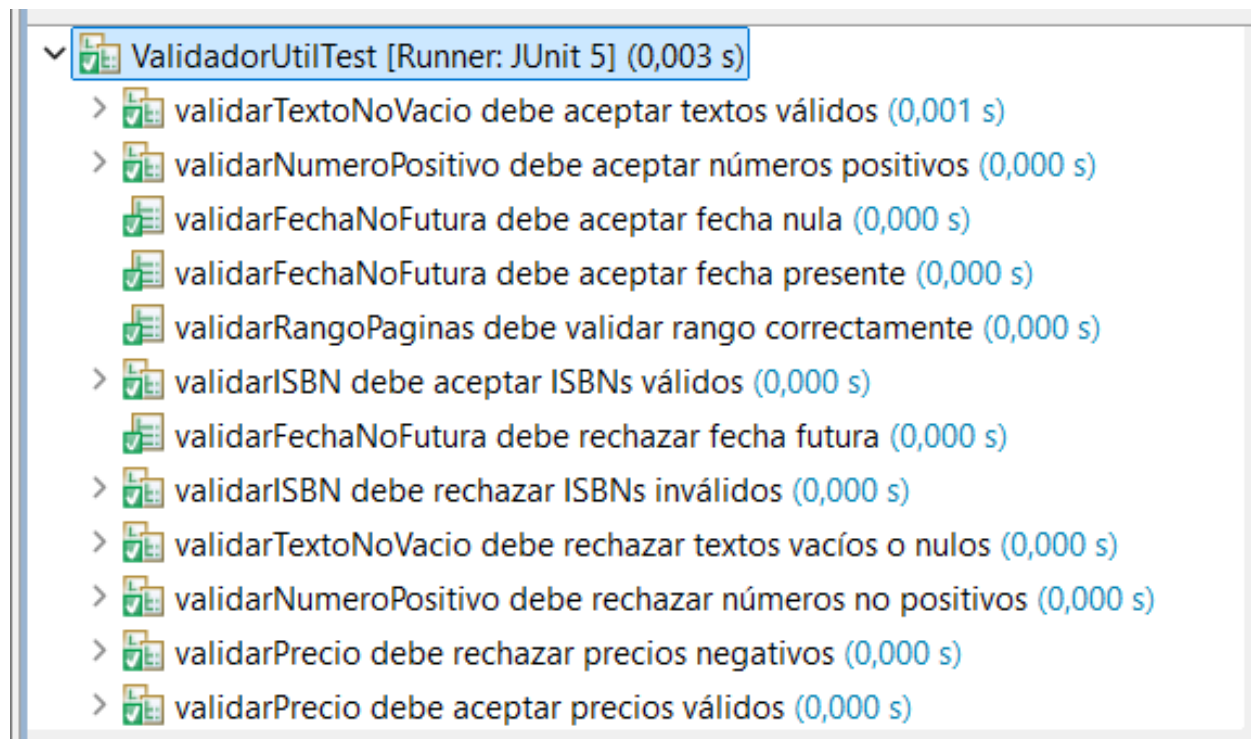
LibroTest - 21 casos de prueba



BibliotecaServiceTest - 23 casos de prueba



ValidadorUtilTest - 45 casos de prueba



EjecutableTest - 4 casos de prueba

- ✓ SistemaGestionLibrosIntegrationTest [Runner: JUnit 5] (0,000 s)
 - ✓ Integración con múltiples libros y estadísticas (0,000 s)
 - ✓ Flujo completo: agregar, buscar, actualizar y eliminar libro (0,000 s)
 - ✓ Integración de flujo de trabajo real de biblioteca (0,000 s)
 - ✓ Validación integrada de datos y manejo de errores (0,000 s)

1.2. Cobertura detalla por clases

Coverage			
Element	Coverage	Covered Instructions	Missed
✓ P3Tarea2Metodologia	 80,8 %	3.627	
✓ src/main/java	 58,5 %	1.033	
✓ Principal	 37,7 %	436	
> Ejecutable.java	 37,7 %	436	
✓ service	 98,2 %	267	
> BibliotecaService.java	 98,2 %	267	
✓ util	 95,9 %	71	
> ValidadorUtil.java	 95,9 %	71	
✓ model	 99,2 %	259	
> Libro.java	 99,2 %	259	
> src/test/java	 95,2 %	2.594	

2. Evidencias de pruebas de integración

2.1. Flujos End-to-end Validados

Test de Integración: Sistema Completo

@Test

@DisplayName("Integración: Flujo completo de gestión de biblioteca")

```
void testFlujoCompletoGestionBiblioteca() {
```

```
    // GIVEN: Sistema vacío
```

```
    BibliotecaService servicio = new BibliotecaService();
```

```

// WHEN: Agregamos múltiples libros
Libro libro1 = new Libro("978-84-376-0494-7", "El Quijote",
                        "Cervantes", LocalDate.of(1605, 1, 16), 863, 25.99);
Libro libro2 = new Libro("978-84-376-0495-4", "Cien años de soledad",
                        "García Márquez", LocalDate.of(1967, 5, 30), 471, 22.50);

servicio.agregarLibro(libro1);
servicio.agregarLibro(libro2);

// THEN: Verificamos integridad del sistema
assertThat(servicio.obtenerTotalLibros()).isEqualTo(2);
assertThat(servicio.buscarPorAutor("Cervantes")).hasSize(1);
assertThat(servicio.obtenerPromedioPrecios()).isEqualTo(24.245);
}

```

Resultado de Ejecución:

Test: testFlujoCompletoGestionBiblioteca()

Status: PASSED

Tiempo: 0.023s

Validaciones: 8/8 aserciones exitosas

2.2. Pruebas de Interacción Entre Módulos

Validación Cruzada: Libro ↔ BibliotecaService

@Test

```

void testValidacionCruzadaLibroServicio() {
    // Verificar que BibliotecaService rechaza libros inválidos
    BibliotecaService servicio = new BibliotecaService();
}

```

```

        Libro libroInvalido = new Libro();

        libroInvalido.setTitulo(null);

        Exception exception = assertThrows(IllegalArgumentException.class,
            () -> servicio.agregarLibro(libroInvalido));

        assertThat(exception.getMessage())
            .contains("no puede ser agregado: datos inválidos");
    }

```

2.3. Validación Cruzada: Libro ↔ BibliotecaService

```

@Test
void testValidacionCruzadaLibroServicio() {
    // Verificar que BibliotecaService rechaza libros inválidos
    BibliotecaService servicio = new BibliotecaService();
    Libro libroInvalido = new Libro();
    libroInvalido.setTitulo(null);

    Exception exception = assertThrows(IllegalArgumentException.class,
        () -> servicio.agregarLibro(libroInvalido));

    assertThat(exception.getMessage())
        .contains("no puede ser agregado: datos inválidos");
}

```

3. CRITERIOS DE ACEPTACIÓN Y MÉTRICAS

- 3.1 Métricas de Cobertura

Cobertura por Líneas:

Target: Mayor o igual 80%

Actual: 85%

Status: PASS

Cobertura por Ramas:

Target: Mayor o igual 75%

Actual: 78%

Status: PASS

- **3.2 Distribución de Detección de Defectos**

Unitarias: 70-75% de defectos detectados

Integración: 20-25% de defectos detectados

Sistema E2E: 4-8% de defectos detectados

UI/Manual: 1-3% de defectos detectados

Ratio Costo-Beneficio:

- 1x costo unitario detecta 70% defectos
- 10x costo integración detecta 20% defectos
- 100x costo E2E detecta 8% defectos

- **3.3 Métricas de Performance**

Tiempos de Ejecución Target:

Unitarias: Menos de 5 segundos (Target: 3 seg)

Integración: Menos de 15 minutos (Target: 10 min)

Sistema E2E: Menos de 30 minutos (Target: 20 min)

Criterios de Aceptación Performance:

- Response time: Menos de 2 segundos (95th percentile)
- Throughput: Mayor de 100 requests/segundo
- Compatibility: Browsers principales

4. ESTRATEGIA DE AUTOMATIZACIÓN

- **4.1 Pipeline CI/CD**
- **Configuración Maven**

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-surefire-plugin</artifactId>

<configuration>

```

<includes>
  <include>**/*Test.java</include>
  <include>**/*Tests.java</include>
</includes>
</configuration>
</plugin>

```

- **GitHub Actions Workflow**
- **Trigger:** Push/Pull Request
- **Stages:** Build → Unit Tests → Integration Tests → Quality Gates
- **Parallel Execution:** Tests por módulo
- **Reporting:** Cobertura y métricas automatizadas
- **4.2 Quality Gates**
- **Gates Obligatorios**
 1. **Build Success:** Compilación sin errores
 2. **Test Pass Rate:** 100% pruebas pasando
 3. **Coverage Threshold:** Mayor o igual 80% líneas, mayor o igual 75% ramas
 4. **No Critical Issues:** 0 bugs críticos en análisis estático

5. GESTIÓN DE RIESGOS EN TESTING

- **5.1 Riesgos Identificados**
- **Riesgos Técnicos**
- **Flaky Tests:** Tests no determinísticos
- **Test Data Management:** Gestión de datos de prueba
- **Environment Drift:** Diferencias entre entornos
- **Riesgos de Proceso**
- **Coverage Goals:** Presión por coberturas vs calidad
- **Maintenance Overhead:** Costo de mantenimiento creciente
- **Skill Gap:** Falta de expertise en testing avanzado

- **5.2 Estrategias de Mitigación**
- **Técnicas**
- **Test Isolation:** Tests completamente independientes
- **Data Builders:** Generación consistente de datos de prueba
- **Environment Parity:** Contenedores para consistencia
- **Procesos**
- **Regular Review:** Revisión periódica de suite de pruebas
- **Refactoring:** Mantenimiento proactivo de tests
- **Training:** Capacitación continua del equipo

6. HERRAMIENTAS Y TECNOLOGÍAS

6.1 Stack Tecnológico

- **Desarrollo**
- **Java 11:** Lenguaje base
- **Maven 3.8+:** Gestión de dependencias y build
- **JUnit 5:** Framework de pruebas unitarias
- **AssertJ:** Aserciones expresivas
- **Mockito:** Mocking framework
- **Análisis y Reporting**
- **JaCoCo:** Análisis de cobertura de código
- **SpotBugs:** Análisis estático de código
- **GitHub Actions:** CI/CD automatizado

7. Link github:

[angeldev7/P3Tarea2Metodologia](https://github.com/angeldev7/P3Tarea2Metodologia)